Stats 21 - HW 2

Quanliang Chen

The questions have been entered into this document. You will modify the document by entering your code.

Make sure you run the cell so the requested output is visible. Download/export it as an HTML file and then print to PDF. Be sure to tag your pages on Gradescope.

Homework is an opportunity to practice coding and to practice problem solving. Doing exercises is where you will do most of your learning.

Using AI or copying someone else's solutions takes away your learning opportunities. It is also academic dishonesty.

Reading

- Think Python: Chapters 6 through 10

Reading is important! Keep up with the reading. I recommend alternating between reading a chapter and then working on exercises.

Additional recommended reading:

- String methods documentation https://docs.python.org/3/library/stdtypes.html#string-methods

Textbook Chapter 5 Problems

Modified Exercise 5.14.2

https://allendowney.github.io/ThinkPython/chap05.html#exercise

In [1]:
```
import time
```

In [2]:
```
time.time()
```

Out[2]:
```
1776739103.264238
```

Write a function `now()` that reads the current time and prints out the time of day in hours, minutes, and seconds, plus the number of days since the epoch. The function does not need to return a value, just print output to the screen.

The result should look like:

"Current time is: 15:25:47. It has been 18370 days since the epoch."

Use `int()` to drop decimal values. You do not need to try to find the date with years and months.

Tip: build your function incrementally. Start by finding how many days have passed since the epoch. (check your answer at the bottom of the page: https://www.epochconverter.com/seconds-days-since-y0) From there find how many hours, etc. Keep in mind the hours will be UTC time.

In [3]:
```
def now():
    t = time.time()
    days = int(t // 86400)
    hours = int((t % 86400) // 3600)
    minutes = int((t % 3600) // 60)
    seconds = int(t % 60)
    print(f"Current time is: {hours}:{minutes}:{seconds}. It has been {da
```

In [4]:
```
now()
```

```
Current time is: 2:38:23. It has been 20564 days since the epoch.
```

Textbook Chapter 6 Problems

Modified Exercise 6.12.4

https://allendowney.github.io/ThinkPython/chap06.html#id2

The Ackermann function, A(m, n), is defined:

$$A(m,n) = \begin{cases} n+1 & \text{if } m = 0 \\ A(m-1,1) & \text{if } m > 0 \text{ and } n = 0 \\ A(m-1, A(m, n-1)) & \text{if } m > 0 \text{ and } n > 0 \end{cases}$$

See http://en.wikipedia.org/wiki/Ackermann_function . Write a function named `ack` that evaluates the Ackermann function. Use your function to evaluate a few test cases. Don't test with $m \geq 4$ as it grows very fast very quickly.

In [5]:
```
def ack(m,n):
    if m == 0:
        return n + 1
    elif m > 0 and n == 0:
        return ack(m-1, 1)
    elif m > 0 and n > 0:
        return ack(m-1, ack(m, n-1))
    else:
        print("Invalid input")
```

In [6]:
```
# test case, should be 61
ack(3, 3)
```

Out[6]: 61

In [7]:
```
# test case, should be 125
ack(3, 4)
```

Out[7]: 125

Modified Exercise based on Think Python 2ed. Chap 6, Section 11, Exercise 6.4

A number, `a`, is a power of `b` if it is divisible by `b` and `a/b` is a power of `b`. Write a function called `is_power` that takes parameters `a` and `b` and returns `True` if a is a power of b. Note: you will have to think about the base case.

In [8]:
```
def is_power(a, b):
    if a % b == 0:
        return is_power(a/b, b)
    elif a == 1:
        return True
    else:
        return False
```

In [9]:
```
is_power(1024, 2)
```

Out[9]: True

In [10]:
```
is_power(6561, 3)
```

Out[10]: True

In [11]:
```
is_power(4374, 3)
```

Out[11]: False

In [12]:
```
is_power(768, 2)
```

Out[12]: False

Modified Exercise 6.12.5

https://allendowney.github.io/ThinkPython/chap06.html#id3

The greatest common divisor (GCD) of a and b is the largest number that divides both of them with no remainder.

One way to find the GCD of two numbers is based on the observation that if `r` is the remainder when `a` is divided by `b`, then `gcd(a, b) = gcd(b, r)`.

As a base case, we can use `gcd(a, 0) = a`.

Write a function called `gcd` that takes parameters `a` and `b` and returns their greatest common divisor.

In [13]:
```
def gcd(a, b):
    if b == 0:
        return a
    else:
        return gcd(b, a % b)
```

In [14]:
```
gcd(21, 7)
```

Out[14]: 7

In [15]:
```
gcd(42, 28)
```

Out[15]: 14

In [16]:
```
gcd(105, 140)
```

Out[16]: 35

Textbook Chapter 7 Problems

Exercise 7.9.2

https://allendowney.github.io/ThinkPython/chap07.html#exercise

Write a function named `uses_none` that takes a word and a string of forbidden letters, and returns `True` if the word does not use any of the forbidden letters.

Here's an outline of the function that includes two doctests. Fill in the function so it passes these tests, and add at least one more doctest.

In [17]:
```
def uses_none(word, forbidden):
    """Checks whether a word avoid forbidden letters.

    >>> uses_none('banana', 'xyz')
    True
    >>> uses_none('apple', 'efg')
    False
    """
    # I use set here for O(1) lookup time
    forbidden_set = set(forbidden)

    for letter in word:
        if letter in forbidden_set:
            return False
    return True
```

In [18]:
```
uses_none('banana', 'xyz')
```

Out[18]: True

In [19]:
```
uses_none('apple', 'efg')
```

Out[19]:
```
False
```

Exercise 7.9.3.

https://allendowney.github.io/ThinkPython/chap07.html#id1

Write a function called `uses_all` that takes a word and a string of letters, and that returns `True` if the word contains all of the letters in the string at least once.

Here's an outline of the function that includes two doctests. Fill in the function so it passes these tests, and add at least one more doctest.

In [20]:
```
def uses_all(word, required):
    """Checks whether a word uses all required letters.

    >>> uses_all('banana', 'ban')
    True
    >>> uses_all('apple', 'api')
    False
    """
    word_set = set(word)
    for letter in required:
        if letter not in word_set:
            return False
    return True
```

In [21]:
```
uses_all('banana', 'ban')
```

Out[21]:
```
True
```

In [22]:
```
uses_all('apple', 'api')
```

Out[22]:
```
False
```

In [23]:
```
uses_all('elephant', 'eha')
```

Out[23]:
```
True
```

Problems from Think Python 2nd Ed. Chapter 9

Exercise 9.1

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 84.

Write and run a script that reads `words.txt` and prints out only the words with more than 20 characters (after stripping whitespace).

In [24]:
```
fin = open("words.txt")
for line in fin:
    if len(line.strip()) > 20:
        print(line.strip())
```

```
counterdemonstrations
hyperaggressivenesses
microminiaturizations
```

Exercise 9.2

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 84.

Write a function called `has_no_e` that returns True if the word doesn't have the letter e. You can use any of Pythons availble string methods.

In [25]:
```
def has_no_e(text):
    return text.find("e") == -1
```

In [26]:
```
has_no_e("hello")
```

Out[26]:
```
False
```

In [27]:
```
has_no_e("quit")
```

Out[27]:
```
True
```

With your function, write a script. The script should read the list of words (`words.txt`), print out the number of words that do not have the letter 'e' and the proportion of words that do not have the letter 'e'

In [28]:
```
fin = open("words.txt")
count = 0
total = 0
for line in fin:
    if has_no_e(line.strip()):
        count += 1
    total += 1
print(f"number of words that do not have the letter 'e': {count}")
print(f"proportion of words that do not have the letter 'e': {count / tot
```

```
number of words that do not have the letter 'e': 37619
proportion of words that do not have the letter 'e': 0.3306409084516946
```

Problems from Think Python 2nd Ed. Chapter 10

Exercise 10.1

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 100

Write a function called `nested_sum` that takes a list of lists of integers and adds up the elements from all of the nested lists. For example:

```
t = [[1, 2], [3], [4, 5, 6]]
nested_sum(t)
21
```

You may want to build the function recursively in case there are many levels of nested lists.

You can assume that all elements in any of the nested lists are numeric.

In [29]:
```
def nested_sum(t):
    total = 0
    for lst in t:
        if isinstance(lst, list):
            total += nested_sum(lst)
        else:
            total += lst
    return total
```

In [30]:
```
t = [1, 2]
nested_sum(t)
```

Out[30]: 3

In [31]:
```
t = [[1, 2], [3], [4, 5, 6]]
nested_sum(t)
```

Out[31]: 21

In [32]:
```
x = [[1, 2, [3]], 4, 5, 6, [7], 8]
nested_sum(x)
```

Out[32]: 36

In [33]:
```
t = [[[1, 2, [3]], [4, [5, 6, [7]], 8]]]
nested_sum(t)
```

Out[33]: 36

Exercise 10.2

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 100

Write a function called `cumsum` that takes a list of numbers and returns the cumulative sum; that is, a new list where the ith element is the sum of the first i + 1 elements from the original list.

For example:

```
t = [1, 2, 3]
cumsum(t)
[1, 3, 6]
```

You can assume that all elements in the lists are numeric and the list does not contain nested lists.

In [34]:
```
def cumsum(t):
    new_t = []
    for element in t:
        if new_t:
            new_t.append(new_t[-1] + element)
        else:
            new_t.append(element)
    return new_t
```

In [35]:
```
cumsum([1, 2, 3, 4])
```

Out[35]:
```
[1, 3, 6, 10]
```

In [36]:
```
cumsum(range(12))
```

Out[36]:
```
[0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66]
```

Exercise 10.6

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

Two words are anagrams if you can rearrange the letters from one to spell the other. Write a function called `is_anagram` that takes two strings and returns `True` if they are anagrams.

You can remove spaces and convert to lowercase using `string.replace(" ","").lower()`

In [37]:
```
def is_anagram(word1, word2):
    word1_clean = word1.replace(" ","").lower()
    word2_clean = word2.replace(" ","").lower()
    return sorted(word1_clean) == sorted(word2_clean)
```

In [38]:
```
is_anagram("hello", "o hell")
```

Out[38]:
```
True
```

In [39]:
```
is_anagram("dormitory" , "dirty room")
```

Out[39]:
```
True
```

In [40]:
```
is_anagram("dormitory" , "dirty rooms")
```

Out[40]:
```
False
```

In [41]:
```
is_anagram("astronomers" , "moon starers")
```

Out[41]: True

Exercise 10.7

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

Write a function called `has_duplicates` that takes a list and returns `True` if there is any element that appears more than once. It should not modify the original list.

You can assume that the list will not have nested lists.

In [42]:
```
def has_duplicates(t):
    return len(t) != len(set(t))
```

In [43]:
```
has_duplicates(['a','b','c'])
```

Out[43]: False

In [44]:
```
has_duplicates(['a','b','b','c'])
```

Out[44]: True

In [45]:
```
has_duplicates(['a','b','c','a'])
```

Out[45]: True

Exercise 10.10

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

To check whether a word is in the word list, you could use the in operator, but it would be slow because it searches through the words in order.

Because the words are in alphabetical order, we can speed things up with a bisection search (also known as binary search). You start in the middle and check to see whether the word you are looking for comes before the word in the middle of the list. If so, you search the first half of the list the same way (perform a bisection search on the first half). Otherwise you search the second half.

Either way, you cut the remaining search space in half. If the word list has 113,809 words, it will take about 17 steps to find the word or conclude that it's not there.

Write a function called `in_bisect` that takes a sorted list and a target word and will returns `True` if the word is in the list and `False` if it's not.

Hint: it's a recursive function.

In [46]:
```
# Use this function. No need to rewrite it.
def make_word_list():
    """Reads lines from a file and builds a list."""
    t = []
    fin = open('words.txt')
    for line in fin:
        word = line.strip()
        t.append(word)
    return t

t = make_word_list()
```

In [47]:
```
# define this function
def in_bisect(word_list, word):

    start, end = 0, len(word_list) - 1
    while start <= end:
        mid = (start + end) // 2

        if word_list[mid] == word:
            return True
        elif word_list[mid] < word:
            start = mid + 1
        else:
            end = mid - 1
    return False
```

In [48]:
```
in_bisect(t, "hello")
```

Out[48]:
```
True
```

In [49]:
```
in_bisect(t, "xyz")
```

Out[49]:
```
False
```

Exercise 10.11

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101

Two words are a "reverse pair" if each is the reverse of the other.

Now that you have the `in_bisect` search, write a script that finds all the reverse pairs in the word list that are 6 letters or longer. (It takes a little bit of time to run.)

In [50]:
```
### ('agenes', 'senega')
### ('animal', 'lamina')
### ...
### ('lamina', 'animal')
### ...
### ('senega', 'agenes')
### Because of the nature of a reverse pair, each pair will appear twice
```

In [51]:
```
pairs = []
for element in t:
    if len(element) >= 6 and in_bisect(t, element[::-1]):
        pairs.append((element, element[::-1]))
print(pairs)
```

```
[('agenes', 'senega'), ('animal', 'lamina'), ('animes', 'semina'), ('degam
i', 'imaged'), ('deified', 'deified'), ('deifier', 'reified'), ('deliver',
'reviled'), ('denier', 'reined'), ('denies', 'seined'), ('denned', 'denne
d'), ('depots', 'stoped'), ('derats', 'stared'), ('dessert', 'tressed'),
('desserts', 'stressed'), ('dewans', 'snawed'), ('dialer', 'relaid'), ('di
aper', 'repaid'), ('dormin', 'nimrod'), ('drawer', 'reward'), ('elides',
'sedile'), ('eviler', 'relive'), ('gelder', 'redleg'), ('halalah', 'halala
h'), ('hallah', 'hallah'), ('imaged', 'degami'), ('lamina', 'animal'), ('l
evins', 'snivel'), ('looter', 'retool'), ('marram', 'marram'), ('nimrod',
'dormin'), ('pupils', 'slipup'), ('recaps', 'spacer'), ('redder', 'redde
r'), ('redips', 'spider'), ('redleg', 'gelder'), ('redraw', 'warder'), ('r
edrawer', 'rewarder'), ('reflet', 'telfer'), ('reflow', 'wolfer'), ('reifi
ed', 'deifier'), ('reifier', 'reifier'), ('reined', 'denier'), ('reknit',
'tinker'), ('reknits', 'stinker'), ('relaid', 'dialer'), ('relive', 'evile
r'), ('remeet', 'teemer'), ('rennet', 'tenner'), ('repaid', 'diaper'), ('r
epaper', 'repaper'), ('repins', 'sniper'), ('retool', 'looter'), ('revile
d', 'deliver'), ('reviver', 'reviver'), ('reward', 'drawer'), ('rewarder',
'redrawer'), ('rotator', 'rotator'), ('sallets', 'stellas'), ('scares', 's
eracs'), ('secret', 'terces'), ('sedile', 'elides'), ('seined', 'denies'),
('selahs', 'shales'), ('selles', 'selles'), ('sememes', 'sememes'), ('semi
na', 'animes'), ('senega', 'agenes'), ('seracs', 'scares'), ('shales', 'se
lahs'), ('skeets', 'steeks'), ('sleeps', 'speels'), ('sleets', 'steels'),
('slipup', 'pupils'), ('sloops', 'spools'), ('snawed', 'dewans'), ('snipe
r', 'repins'), ('snivel', 'levins'), ('snoops', 'spoons'), ('spacer', 'rec
aps'), ('speels', 'sleeps'), ('spider', 'redips'), ('spirts', 'strips'),
('spools', 'sloops'), ('spoons', 'snoops'), ('sports', 'strops'), ('sprit
s', 'stirps'), ('stared', 'derats'), ('steeks', 'skeets'), ('steels', 'sle
ets'), ('stellas', 'sallets'), ('stinker', 'reknits'), ('stirps', 'sprit
s'), ('stoped', 'depots'), ('stressed', 'desserts'), ('strips', 'spirts'),
('strops', 'sports'), ('struts', 'sturts'), ('sturts', 'struts'), ('teeme
r', 'remeet'), ('telfer', 'reflet'), ('tenner', 'rennet'), ('terces', 'sec
ret'), ('terret', 'terret'), ('tinker', 'reknit'), ('tressed', 'dessert'),
('warder', 'redraw'), ('wolfer', 'reflow')]
```